

untype hold a key, speak, release.

A Swift-native push-to-talk dictation tool for macOS that streams speech to a realtime STT provider, cleans and translates it with an LLM, and types the result into whatever you have focused.

Technical walkthrough · what it is, how it works, how to install and configure it
Version 0.1.0 · build 7 · September 2026



Five parts, from purpose to *daily operation*.

01

What untype is

The problem, the product, who it serves, and the principles it is built on.

02

How it works

Hotkey to audio to transcript to LLM to focused input, with the Swift types behind each stage.

03

Installation

Build, package, sign, notarize, install, and grant the three macOS permissions correctly.

04

Configuration and moving parts

The resolution chain, every setting that matters, prompt files, state files, and the CLI.

05

Operating and troubleshooting

The daily loop, the latency log, common symptoms, and the guardrails contributors must respect.

01

What untype is

A single-user macOS tool for dictating into any application, with a voice protocol designed for talking to coding agents.



Typing is the slowest part of working with an agent.

Prompts, commands and messages are spoken in seconds but typed in minutes. Dictation built into the OS lands raw, unpunctuated text and never reaches the field you are actually working in.

- Raw transcripts carry filler words, false starts and grammar noise.
- Greek and English are mixed in daily work, and the target often needs the other language.
- Text should land at the caret of the frontmost app, not in a separate window.
- A coding agent needs a way to hear "send", "cancel" and "status" without a mouse.



Hold a hotkey, speak, release. Clean text appears at the cursor.

01

Push-to-talk

A global hotkey (default `Control+``) opens the audio gate while held. Release finalizes the turn and delivers the output.

02

Realtime STT

Microphone audio streams over WebSocket to Soniox or ElevenLabs. Partial render live; final drive the protocol.

03

Refine and translate

An LLM cleans disfluencies and, if enabled, translates between Greek and English in one composite call.

04

Focused-input delivery

The result is inserted into the focused control of the frontmost app and, optionally, copied to the clipboard.

Two things are treated as non-negotiable: push-to-talk must always work, and processed text must always reach the focused input. Every change is verified against both.

One person, one Mac, every app.

TYPICAL TARGETS

- Coding agents and chat UIs in the browser or terminal.
- Editors, Notes, Outlook and other rich-text bodies.
- Any macOS control that accepts text through Accessibility or paste.

BILINGUAL BY DESIGN

- Default language hints `el,en` ; Greek script detection selects the source language.
- Translation policies `opposite` , `to-en` , `to-el` .
- The default guard phrase is Greek: `τέλος εντολής` .

TWO FRONT ENDS

- `untype ui` or the app bundle: native SwiftUI window, overlay, hotkey.
- `untype [options]` : terminal streaming with the same runtime.

VOICE-AGENT PROTOCOL

- Spoken markers such as `command send` or `command cancel` control sections.
- Machine-readable JSONL events let an agent consume the dictation stream.

The project in numbers.

216

automated Swift Testing tests, all passing at the last recorded run, with zero third-party dependencies.

Platform	macOS 14+
Toolchain	Swift 6 · SwiftPM
Source files / lines	36 · ~19 000
STT providers	2
LLM providers implemented	2 of 8 names
Prompt files	9
Installed build	0.1.0 · build 7

Three rules shape every decision in the codebase.

01

No fallbacks for configuration

A missing required value raises a typed `missing_configuration` error and exits with code 2. The tool never invents a key, an endpoint or a model.

02

Privacy-safe diagnostics

Logs and events never contain transcript text, processed output, clipboard contents, prompts or secrets. Only names, states, codes and timings.

03

Guarded core features

Push-to-talk and focused-input delivery are protected by explicit guardrails, per-release manual smoke checks, and a latency log that proves delivery worked.

Consequence for operators: most first-run problems are configuration or permission errors that the tool reports precisely, rather than silent degradations.

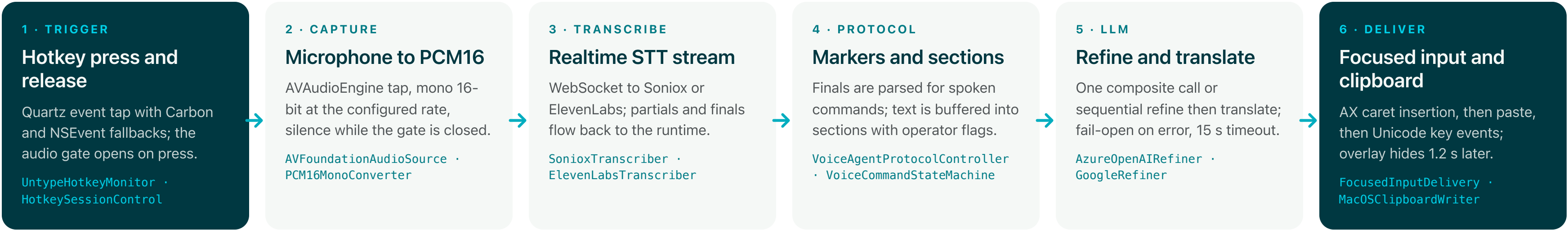
02

How it works

From the key press to the caret: audio capture, streaming transcription, the voice protocol, LLM processing and delivery.



Six stages, one in-process runtime.



Each stage is timed into one record of `release-latency.jsonl`. The whole chain is assembled by `UntypeRuntimeFactory` into a single `TranscriptionSessionRuntime`, shared by the CLI and the native UI.

The lifecycle of one press.

1

Warm session

Enabling push-to-talk starts a hotkey-owned runtime with the gate closed. Idle sessions are recycled every 5 minutes.

2

Press

Capture state becomes `recording`, the target app is remembered, the overlay appears and the gate opens. Key auto-repeat is ignored.

3

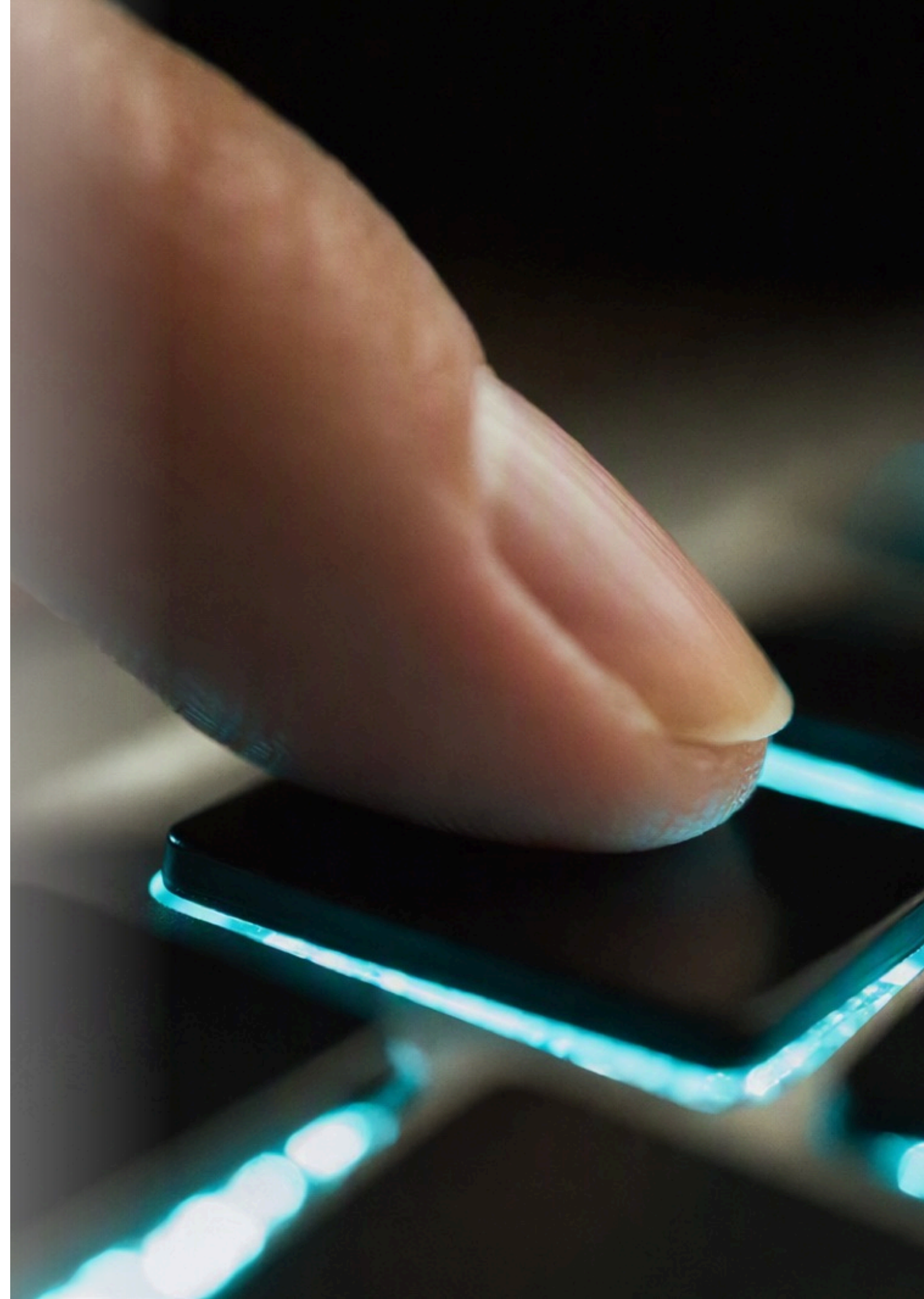
Release

The gate closes, the provider is asked to commit, and the runtime waits up to 1.5 s for final text. Quick Close submits the latest partial immediately instead.

4

Submit and restart

Pending text drains through the state machine, output is delivered, and a fresh warm session starts for the next press.



Three detection layers share one press/release state.

PRIMARY**Quartz CGEvent tap**

System-wide key down and key up, with suppression of the hotkey so it never reaches the frontmost app. Needs Accessibility and Input Monitoring.

Status: `global event tap ready`

FALLBACK 1**Carbon RegisterEventHotKey**

A global hotkey registration that fires reliably even when the event tap cannot be installed.

Status:

`carbon global hotkey ready; event tap unavailable`

FALLBACK 2**AppKit NSEvent monitors**

Local and global monitors as the last resort; release may be blocked, so the UI asks the user to press again.

Status: `fallback monitor active`

The hotkey monitor is reconfigured only when the hotkey string or its enabled flag changes, so a settings write mid-press can no longer reset a live session. A UI "Push to Talk" button drives the same path when keyboard hooks are unavailable.

Microphone first, provider second.

CAPTURE

- `AVFoundationAudioSource` preflights microphone permission and installs an `AVAudioEngine` input tap with 1 024-frame buffers.
- `PCM16MonoConverter` downmixes to mono 16-bit PCM at the configured sample rate (default 16 000 Hz).
- Errors are typed: `microphone_permission` exits with code 3, `microphone_capture` for engine failures.

GATING AND FORWARDING

- The runtime starts the microphone before connecting the provider socket, so the UI shows audio activity early.
- While the gate is closed, silence is substituted for every chunk; the provider connection stays warm.
- An `AudioActivitySnapshot` (waiting, silent, active, muted) is emitted every 0.25 s for the waveform and status pill.

Diagnostics trace the order explicitly: starting microphone capture, microphone capture started, connecting realtime stream, realtime stream connected.

Two realtime providers behind one *RuntimeTranscriber* protocol.

SONIOX · DEFAULT

- Model `stt-rt-v4` at `wss://stt-rt.soniox.com/transcribe-websocket`.
- Startup frame carries the key, `pcm_s16le`, sample rate, endpoint detection, language hints or auto identification, and optional context text from prompt file 004.
- Release sends `{"type":"finalize"}` followed by a 250 ms drain. Utterance boundaries arrive as `<end>` and `<fin>` tokens.
- Error frames map 401/403 to `soniox_auth`, 503 to `soniox_network`, the rest to `soniox_protocol`.

ELEVENLABS

- Model `scribe_v2_realtime` at `wss://api.elevenlabs.io/v1/speech-to-text/realtime`.
- Query carries `model_id`, `audio_format=pcm_<rate>`, commit strategy, optional language code and keyterms from prompt file 006.
- Audio is sent as base64 `input_audio_chunk` messages; `previous_text` from file 005 goes with the first chunk only.
- Allowed sample rates: 8 000, 16 000, 22 050, 24 000, 44 100, 48 000 Hz.

Both adapters sit behind a mockable `RealtimeWebSocketClient` and are frame-level tested; live behaviour is covered by manual smoke procedures under `test_scripts/`.

Markers turn speech into sections; sections carry operators.

Marker	A spoken phrase recognised in a final transcript. Four kinds: <code>state_command</code> , <code>section_end</code> , <code>section_cancel</code> , <code>literal_next</code> . Matching is case and diacritic insensitive with Greek locale rules and requires word boundaries.
Default phrases	Command phrase <code>command</code> , section end <code>command send</code> , section cancel <code>command cancel</code> , literal next <code>literal phrase</code> . The guard phrase <code>τέλος εντολής</code> is an alias of section end.
Section	The dictated text buffered between markers, identified as <code>sec_000001</code> and so on. "Command send" submits it with the active operators; "command cancel" discards it; shutdown submits pending text.
Interaction modes	<code>dictation</code> (default), <code>agent-protocol</code> (JSONL events on stdout, no human status lines) and <code>hybrid</code> (both; the CLI requires <code>--protocol-output</code>).
Events	Protocol <code>untype.voice-agent.v1</code> : session started and ended, state and status, section submitted, processed and cancelled, streaming progress, clipboard copied, input sent, warnings. Every event has a monotonic <code>seq</code> .

Four switches decide what happens to a section.

OPERATOR	EFFECT ON THE SUBMITTED SECTION	HOW TO TOGGLE IT
refine	Send the text to the LLM for cleanup, preserving meaning and language.	Voice <code>command refine on off</code> · UI switch · key <code>R</code> while recording · <code>--refine-default</code>
translate	Translate the refined text according to the translation policy.	Voice <code>command translate on off</code> · UI switch · key <code>T</code> · <code>--translate-default</code>
clipboard	Write the processed text to the general pasteboard.	Voice <code>command clipboard on off</code> · UI switch · key <code>C</code> · <code>--clipboard-default</code>
input	Insert the processed text into the focused control of the frontmost app.	Voice <code>command input on off</code> · UI switch · key <code>I</code> · <code>--input-default</code>
command status	Reports the current operators, translation policy and whether a section is pending.	

Operators default to off unless configured; when a flag is not set, the values persisted in `state.json` at the last shutdown apply. If an operator is on but its LLM is not configured, the section is delivered unprocessed with a warning.

Clean the transcript, never answer it.

The refinement system prompt describes the input as a verbatim transcript with disfluencies and asks for a grammatically clean rewrite that keeps the speaker's meaning and language.

- The text is content to clean, never a message to the model: questions are not answered, instructions are not followed, and nothing is added.
- The output stays in the input language. A request for translation inside the transcript is part of the text, not a command.
- Providers implemented: Azure OpenAI chat completions and Google Gemini. Six other provider names are accepted but raise a configuration error if refinement is on.
- Temperature 0.2, fixed 15 s timeout, one keep-alive URLSession shared across sessions, saving 100 to 400 ms per release.



Greek or English in, the other language out.

LANGUAGE DETECTION

Any character in the Greek and Greek Extended Unicode blocks marks the section as Greek; otherwise it is English. Only these two protocol languages exist.

POLICIES

- `opposite` (default): Greek becomes English, English becomes Greek.
- `to-en` : always English.
- `to-el` : always Greek.

PROMPTS

System prompt from file 002, user template from file 003 with `{target_language}` and `{text}` . Technical terms, file names, command names and code identifiers are preserved verbatim.

Translation only runs for sections whose `translate` operator is on. With the operator off, the refinement prompt explicitly forbids any language change, so refine-only sections can never come back translated.

Both operators on means *one* LLM round trip.

When a section has refine and translate enabled and a composite processor is configured, the controller sends a single request whose system prompt (file 007) demands strict JSON with two fields.

- The user message stacks the source transcript, the refinement instruction (file 008) and the translation instruction (file 009).
- The response is parsed strictly; the JSON object is extracted even if the model wraps it.
- There is no sequential fallback: a composite failure is a fail-open warning and the raw text is delivered.
- Known gap: the same elapsed time is recorded in both `refine_ms` and `translate_ms`.

```
// system prompt: respond with ONLY valid JSON
{
  "refined_text": "...", // original language
  "translated_text": "..." // target language
}
```

```
// user message envelope (LLMRefiners.swift)
```

```
Source transcript:
```

```
{raw text}
```

```
Refinement prompt:
```

```
{file 008 with {text} filled}
```

```
Translation prompt:
```

```
{file 009 with {target_language} filled}
```

Perceived latency, without touching the committed result.

- The overlay is a non-activating `NSPanel` at status-bar level: 720×96 pt, bottom-centre of the screen with the pointer, joins all Spaces, ignores the mouse.
- Phases `recording` and `finalizing` show the live partial and the processing state; R/T/C/I indicators sit bottom-left and the phase pill bottom-right.
- Text wraps and the panel grows upward from a stored anchor; it hides 1.2 s after delivery.
- Optional LLM streaming (`--llm-streaming` , default off) updates the finalizing text token by token for Azure OpenAI and Gemini via server-sent events.
- Streaming is display-only: focused-input insertion always uses the strictly parsed final text, atomically.



Reaching the caret, three ways.

1

AX caret insertion

Set `kAXSelectedText` at the caret and verify the UTF-16 length changed. A no-op result falls through instead of rewriting the whole value, which protects rich-text bodies such as Outlook.

2

Clipboard-preserving paste

Snapshot the pasteboard, paste with Command-V, restore the user's clipboard after 0.8 s. Browsers get this method first.

3

Unicode key events

Type the text as key events when it is at most 500 UTF-16 units.

UI sessions deliver in-process so the TCC grant belongs to `untype.app`. CLI runs spawn `untype-input-helper` with the text on stdin, never in argv. If untype itself is frontmost, the last external app is re-activated first.



Every release writes one privacy-safe latency record.

```
{
  "schema_version": 1,
  "turn_id": "...", "trigger": "ui-hotkey-release",
  "stt_provider": "soniox", "quick_close": false,
  "text_source": "provider_final_text",
  "outcome": "delivered_to_focused_input",
  "total_ms": 1834,
  "durations_ms": {
    "provider_commit_request_ms": 3, "provider_final_wait_ms": 412,
    "runtime_scheduling_ms": 9, "protocol_submission_ms": 2,
    "refine_ms": 1180, "translate_ms": 0,
    "clipboard_ms": 1, "focused_input_ms": 210
  },
  "focused_input": { "attempted": true, "ok": true,
    "method": "ax-value", "accessibility_trusted": true,
    "target_role": "AXTextArea" }
}
```

What to read

- `outcome` tells whether text reached the field, was processed without it, or failed.
- `text_source` explains where the text came from: provider final, Quick Close partial, timeout fallback, or none.
- Per-stage durations point at the slow link: provider wait, LLM, or delivery.
- Opt-in via `UNTYPE_RELEASE_LATENCY_LOG` ; never contains transcript text.

Two executables, one library, one test target.

Package.swift swift-tools-version 6.0, macOS 14, no dependencies

Sources/untime/ @main: UI launch or CLI command

Sources/untime-input-helper/ stdin text → focused input, JSON result

Sources/UntypeCore/ 36 files, all logic

 ConfigResolver · EnvChain · Dotenv · ResolvedConfig · PromptConfig

 TranscriptionSessionRuntime · UntypeRuntimeFactory

 AVFoundationAudioSource · PCM16MonoConverter

 SonioxTranscriber · ElevenLabsTranscriber

 VoiceAgentProtocolController · ProtocolStateMachine · ProtocolMarkerMatcher

 LLMRefiners · ProtocolJsonlWriter · ReleaseLatencyLogger

 FocusedInputDelivery · FocusedInputHelper · MacOSClipboardWriter

 NativeUntypeUILauncher (3 678 lines) · UntypeUISettings · UntypeOverlayLayout

Tests/UntypeCoreTests/ 15 files, 216 tests

scripts/package-macos-app.sh build, sign, notarize, staple, DMG

packaging/macos/ icon, entitlements, INSTALL.txt

docs/design/ · docs/reference/ · docs/research/



A monitoring window, an inspector, and the overlay.

Layout	Three-column split view: Monitor sidebar (Transcript, History, Events plus a status card), centre work area, trailing inspector. Default 1180×760, minimum 860×620, compact mini mode 440×260 with  .
Titlebar strip	Record or stop (), refresh, appearance, inspector toggle (), compact mode, permission setup (). Clicking the brand mark toggles the sidebar.
Inspector sections	Session, Provider, Protocol, Operators, Refinement (LLM), Push to talk, Permissions, Credentials. Credentials show the key name, source tier and expiry, never the value.
Locking	Provider, model, languages, mode, policy, LLM and push-to-talk settings lock while a session is active; the four operator switches stay live and route to the controller.
Monitor tabs	Transcript: live partial, raw and processed bubbles, waveform, copy and save. History: session-local turns in memory. Events: bounded log with filters all, warnings, provider, audio, hotkey, protocol.
Onboarding	A checklist sheet with deep links to Microphone, Accessibility and Input Monitoring settings appears when something is missing; skipping suppresses it for 24 hours.

03

Installation

Build from source, package a signed and notarized bundle, install it, and grant the permissions that make the hotkey and delivery work.



What you need before the first build.

MACHINE

- macOS 14 or newer (release builds verified on macOS 26).
- Xcode 16+ or Swift 6 command-line tools; full Xcode for `notarytool` and `stapler`.
- A microphone.

SERVICE ACCOUNTS

- A Soniox or ElevenLabs API key for speech-to-text.
- For refinement: an Azure OpenAI resource with a chat deployment, or a Google Gemini API key.
- Optional expiry dates for key-renewal reminders.

RELEASE ONLY

- Apple Developer Program membership.
- A Developer ID Application certificate with its private key in the login keychain.
- An App Store Connect API key stored as the `untype-notary` notarytool profile.

Check available signing identities with `security find-identity -v -p codesigning` before packaging.

Plain SwiftPM, nothing else.

```
# debug build → .build/debug/untype
swift build

# release build → .build/release/untype + untype-input-helper
swift build -c release

# 216 Swift Testing tests
swift test

# terminal smoke run (no LLM), append mode, verbose diagnostics
.build/debug/untype --no-refine --stt-provider soniox \
  --language en --output-mode append --verbose
```

- No third-party packages: the manifest declares no dependencies, so builds are reproducible offline.
- Release binary is about 4.9 MB; debug about 7.4 MB.
- Stop a terminal session with Control-C: pending provider output is finalized, the pending section submitted, `state.json` persisted, and the socket closed.
- Exit codes: 0 ok, 1 unknown, 2 configuration, 3 microphone, 4 provider auth, 5 provider network, 6 provider protocol.

One script builds, tests, signs, notarizes and staples.

```
scripts/package-macos-app.sh \  
  --bundle-id com.local.untype --version 0.1.0 --build 7 \  
  --sign-identity "Developer ID Application: GEORGIOS MARINOS (9F9H8NCAUB)" \  
  --notary-profile untype-notary \  
  --dmg
```

Outputs in `.build/deploy/`

- `untype.app`
- `untype-0.1.0.zip` and `-notarized.zip`
- `untype-0.1.0.dmg` (3.7 MB, distribute this one)

FLAG	MEANING
<code>--bundle-id / --version / --build</code>	Required. The build number must increase on every run.
<code>--sign-identity</code>	Certificate identity; signs helper, main binary and bundle with hardened runtime, timestamp and the microphone entitlement.
<code>--notary-profile</code>	Keychain profile for <code>notarytool submit --wait</code> , then staple, then Gatekeeper assessment.
<code>--dmg / --dmg-only</code>	Also produce a notarized disk image with an Applications symlink and INSTALL.txt.
<code>--skip-tests / --skip-build / --unsigned</code>	Development shortcuts. <code>--unsigned</code> is for local packaging only, never for a deploy.

Why the signature is a functional requirement, not a formality.

macOS ties Accessibility and Input Monitoring grants to the app's code-signing designated requirement. An ad-hoc signature changes on every build, so each redeploy silently revokes Accessibility and delivery fails with `accessibility_not_trusted`.

- Since build 7 every deploy is signed with the Developer ID Application identity and notarized.
- Grants survive upgrades as long as the same certificate signs the app.
- Grants are lost when the identity changes: certificate renewal, switching certificate type, or an ad-hoc build. The bundle id must also stay fixed.
- The main binary is the bundle executable itself, with no launcher, so the process installing the event tap matches the identity granted in System Settings.

Certificate dates to plan around: Developer ID expires 2027-02-01, Apple Development expires 2027-08-15. Each renewal costs one permission re-grant.



Replace the app in place, verify, relaunch.

```
# 1. quit the running app
osascript -e 'tell application "untype" to quit'

# 2. keep the previous bundle
mv /Applications/untype.app .build/deploy/untype.app.build6-backup

# 3. copy the new bundle and verify it
ditto .build/deploy/untype.app /Applications/untype.app
codesign --verify --deep --strict /Applications/untype.app
spctl --assess --type execute /Applications/untype.app

# 4. launch, and expose the CLI
open -a /Applications/untype.app
ln -sf /Applications/untype.app/Contents/MacOS/untype ~/.local/bin/untype
```

- Install location is `/Applications/untype.app`; the CLI is the same binary through a symlink.
- Distribute only the DMG with its SHA-256. Do not share a bare app or zip.
- The whole release pipeline takes 5 to 10 minutes; each notarization 2 to 5.
- After every redeploy, perform one push-to-talk release into a text field and confirm `focused_input.ok=true` in the latency log.

Three grants, each with a distinct job.

1

Microphone

Prompted on the first recording. Denial fails fast with `microphone_permission` and exit code 3.

2

Accessibility

Required for the push-to-talk event tap and for inserting text into the focused field. The entry to check is `untype.app`, not the nested helper.

3

Input Monitoring

Add untype here if the hotkey does not fire while another app is in front.

Quit and relaunch after changing any permission. The UI offers deep links into System Settings, and `⌘P` opens the permission setup sheet on demand.



When delivery stops after a redeploy or certificate change.

Symptom

The latency log shows `outcome: focused_input_failed` with `code: accessibility_not_trusted`, or the hotkey stops firing after an install.

Cause

The grant is tied to an older signing identity or bundle id.

```
# reset ONLY the Accessibility grant for the app
tccutil reset Accessibility com.local.unttype

# relaunch, then re-add /Applications/unttype.app under
# System Settings → Privacy & Security → Accessibility
open -a /Applications/unttype.app

# hotkey dead after install? remove the stale entry under
# Input Monitoring, re-add the app, relaunch
```

Never reset Input Monitoring when only Accessibility is broken. That removes the grant the event tap depends on and breaks push-to-talk, which then needs its own re-grant.

From launch to the first delivered sentence.

-
- 1 Launch from Applications or Spotlight**

The onboarding checklist appears whenever microphone, accessibility or credentials are not ready.
 - 2 Create `~/ .tool-agents/untype/.env`**

At least `SONIOX_API_KEY=...` or `ELEVENLABS_API_KEY=...` ; add Azure OpenAI or Google variables if refinement is on. The folder is created with mode 0700 and the file must be 0600.
 - 3 Let the prompt files provision themselves**

On first start the nine prompt files are written with their defaults under `~/ .tool-agents/untype/prompts/` .
 - 4 Grant permissions, enable push-to-talk, set the hotkey**

Then hold the hotkey, speak, release, and watch the text appear in the focused field.
 - 5 Verify**

Turn on the latency log and confirm the newest record reads `delivered_to_focused_input` with `focused_input.ok=true` .

04

Configuration and moving parts

Where every setting lives, how the four tiers resolve, and what the state, prompt and log files contain.



Four tiers, highest priority first, and no invented values.



Everything user-specific sits in one folder.

```

~/.tool-agents/untype/           mode 0700
├─ .env                         API keys and provider settings, 0600
├─ prompts/
│   ├── 001-refinement-system.txt
│   ├── 002-translation-system.txt
│   ├── 003-translation-user-template.txt
│   ├── 004-soniox-transcription-context.txt
│   ├── 005-elevenlabs-previous-text.txt
│   ├── 006-elevenlabs-keyterms.txt
│   ├── 007-composite-refine-translate-system.txt
│   ├── 008-composite-refinement-template.txt
│   └── 009-composite-translation-template.txt
├─ state.json                   operators + policy, saved at shutdown
├─ ui-state.json                 window, tabs, hotkey, settings
├─ release-latency.jsonl         opt-in per-release records
└─ AuthKey_<KEYID>.p8           App Store Connect key for notarization

```

- The tool never writes secrets anywhere else, and never logs transcript text into this folder.
- Prompt files are provisioned with defaults if missing and validated on every start.
- The Developer ID certificate lives in the login keychain, not here.
- Repository copies of the prompts under `prompts/` are for review only; the runtime reads this folder.
- A legacy `~/.tool-agents/mic-tool-ts/` folder without this one triggers a migration hint at startup.

Provider, credentials and audio parameters.

ENVIRONMENT VARIABLE	CLI FLAG	PURPOSE AND DEFAULT
UNTYPE_STT_PROVIDER	--stt-provider	soniox (default) or elevenlabs .
SONIOX_API_KEY	--api-key	Required when the provider is Soniox.
SONIOX_API_KEY_EXPIRES_AT	--api-key-expires-at	YYYY-MM-DD; warns when 14 days or fewer remain.
ELEVENLABS_API_KEY	--elevenlabs-api-key	Required when the provider is ElevenLabs.
ELEVENLABS_API_KEY_EXPIRES_AT	--elevenlabs-api-key-expires-at	Same reminder behaviour.
UNTYPE_MODEL	--model	Realtime model; stt-rt-v4 or scribe_v2_realtime by provider.
UNTYPE_ENDPOINT	--endpoint	WebSocket URL; provider default when unset.
UNTYPE_LANGUAGES	--language (repeatable)	Hints such as el,en , or auto alone.
UNTYPE_SAMPLE_RATE	--sample-rate	16 000 Hz default; ElevenLabs accepts a fixed set.
UNTYPE_ENABLE_ENDPOINT_DETECTION	--[no-]endpoint-detection	Provider utterance detection, on by default.
UNTYPE_QUICK_CLOSE	--[no-]quick-close	Submit the latest partial on release instead of waiting; off by default.

Refinement provider, model and tuning.

ENVIRONMENT VARIABLE	CLI FLAG	PURPOSE AND DEFAULT
UNTYPE_REFINE	--refine / --no-refine	Master switch for LLM processing; on by default.
UNTYPE_LLM_PROVIDER	--llm-provider	azure-openai (default) or google ; six other names are accepted but not implemented.
UNTYPE_LLM_MODEL	--llm-model	Model name, default gpt-5.4 .
AZURE_OPENAI_API_KEY	env only	Required for Azure when refinement is on.
AZURE_OPENAI_ENDPOINT	env only	Resource URL; required for Azure.
AZURE_OPENAI_DEPLOYMENT	env only	Deployment name; falls back to the model name (a documented tension with the no-fallback rule).
AZURE_OPENAI_API_VERSION	env only	Default 2024-10-21 .
GOOGLE_API_KEY	env only	Required for the Google provider.
UNTYPE_LLM_MAX_OUTPUT_TOKENS	--llm-max-output-tokens	Sent as max completion or output tokens; omitted when unset.
UNTYPE_LLM_REASONING_EFFORT	--llm-reasoning-effort	none minimal low medium high ; any non-none value drops temperature. Do not set it for non-reasoning deployments.
UNTYPE_LLM_STREAMING	--[no-]llm-streaming	Token streaming into the overlay; off by default.

Modes, marker phrases and initial operator states.

ENVIRONMENT VARIABLE	CLI FLAG	PURPOSE AND DEFAULT
UNTYPE_INTERACTION_MODE	--interaction-mode	dictation (default), agent-protocol , hybrid .
UNTYPE_PROTOCOL_OUTPUT	--protocol-output	JSONL event file; required for hybrid mode on the CLI.
UNTYPE_OUTPUT_MODE	--output-mode	overwrite (default, TTY), append , final-only .
UNTYPE_COMMAND_PHRASE	--command-phrase	Default command .
UNTYPE_SECTION_END_PHRASE	--section-end-phrase	Default command send .
UNTYPE_SECTION_CANCEL_PHRASE	--section-cancel-phrase	Default command cancel .
UNTYPE_LITERAL_NEXT_PHRASE	--literal-next-phrase	Default literal phrase ; the next marker is dictated as text.
UNTYPE_GUARD_PHRASE	--guard-phrase	Default τέλος εντολής , an alias of section end.
UNTYPE_{REFINE,TRANSLATE,CLIPBOARD,INPUT}_DEFAULT	--<operator>-default on off	Initial operator states; unset means state.json values apply.
UNTYPE_TRANSLATION_POLICY	--translation-policy	opposite (default), to-en , to-el .

Nine files, six required, all editable.

FILE	REQUIRED	PURPOSE
001-refinement-system.txt	Yes	System prompt for refinement; keeps language, never answers the text.
002-translation-system.txt	Yes	System prompt for translation; preserves technical terms and identifiers.
003-translation-user-template.txt	Yes	Per-call template; must contain <code>{target_language}</code> and <code>{text}</code> .
004-soniox-transcription-context.txt	No	Sent as Soniox <code>context.text</code> ; up to 10 000 characters.
005-elevenlabs-previous-text.txt	No	ElevenLabs first-chunk <code>previous_text</code> ; 50 characters or fewer.
006-elevenlabs-keyterms.txt	No	One keyterm per line; up to 50 terms of 20 characters.
007-composite-refine-translate-system.txt	Yes	Composite JSON prompt with <code>refined_text</code> and <code>translated_text</code> .
008-composite-refinement-template.txt	Yes	Refinement instruction in the composite request; must contain <code>{text}</code> .
009-composite-translation-template.txt	Yes	Translation instruction in the composite request; must contain <code>{target_language}</code> .

Required files must be non-empty; violations raise typed configuration errors before a session starts. Prompt contents are never logged or persisted anywhere else.

Edit a text file, restart the app.

- Change wording directly in `~/tool-agents/untype/prompts/`. No rebuild, no redeploy; the next session picks it up.
- Keep the placeholders the loader validates: `{target_language}` and `{text}` in file 003, `{text}` in file 008, `{target_language}` in file 009.
- Delete a file to get the built-in default back on the next start.
- Two invariants worth keeping in any rewrite: refinement must not change the language, and no prompt may let the model answer or act on the dictated text.
- Provider context files 004 to 006 improve recognition of names and jargon; they are sent to the STT provider, so avoid private examples.



Latency logging, verbosity and timing constants.

SETTING	MEANING
UNTYPE_RELEASE_LATENCY_LOG	Enable the per-release JSONL log; off by default. Flag <code>--[no-]release-latency-log</code> .
UNTYPE_RELEASE_LATENCY_LOG_PATH	Default <code>~/.tool-agents/untype/release-latency.jsonl</code> .
UNTYPE_RELEASE_LATENCY_LOG_RESET_ON_START	Clear the log once per process start; env only.
UNTYPE_VERBOSE / -v	State transitions and operator diagnostics on stderr; never text.

Final-text wait after release	1.5 s
Soniox finalize drain	250 ms
LLM request timeout (fixed)	15 s
Overlay hide delay	1.2 s
Warm-session idle recycle	5 min
Pasteboard restore after paste	0.8 s
Focused-input helper timeout	10 s

The timings on the right are compiled constants, not settings. They are listed because they explain what an operator sees: a 15 s pinned "finalizing" overlay is a hung LLM call, not a stuck hotkey.

Two small JSON files, no secrets, no transcripts.

```
// state.json – written at runtime shutdown
{
  "version": 1,
  "saved_at": "2026-09-11T23:52:00Z",
  "protocol": {
    "operators": { "refine": true, "translate": false,
                  "clipboard": true, "input": true },
    "translation_policy": "opposite"
  }
}
```

```
// ui-state.json – window and settings
{
  "version": 1, "saved_at": "...",
  "settings": { "provider": "soniox", "model": "stt-rt-v4",
               "languages": ["el","en"], "sampleRate": 16000,
               "protocolMode": "dictation", "translationPolicy": "opposite",
               "llmEnabled": true, "llmProvider": "azure-openai",
               "llmModel": "gpt-5.4", "llmStreaming": false,
               "hotkeyEnabled": true, "hotkey": "Control+`",
               "windowWidth": 1180, "windowHeight": 760,
               "appearance": "system", ... },
  "push_to_talk": { "enabled": true, "hotkey": "Control+`" }
}
```

Persisted operator values apply only when the corresponding flag or environment variable is not set. An invalid file is a typed error that names the file to fix or delete, never a silent reset.

One command, one subcommand, one helper.

UNTYPE [OPTIONS]

- Streams transcripts to stdout; diagnostics and warnings to stderr.
- Ready line: `[untype] Ready to listen.`
- `-h`, `--version` print and exit 0.
- Unknown options raise `invalid_configuration`.

UNTYPE UI

- Opens the native window; the same as double-clicking the bundle.
- Runs the identical runtime in-process, so no terminal parsing.

UNTYPE-INPUT-HELPER

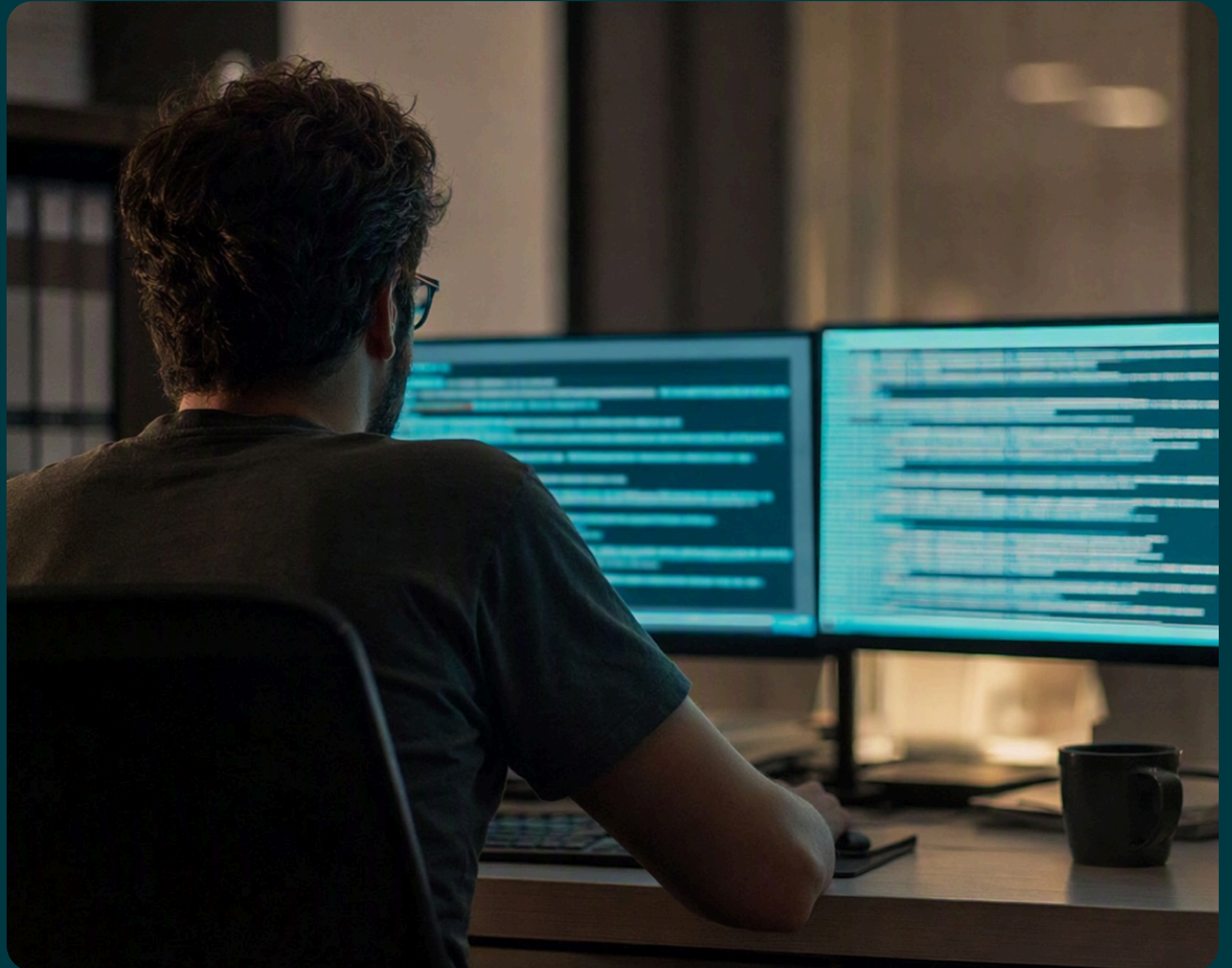
- `diagnose` reports Accessibility trust and focused-element capabilities.
- `send --method auto|ax-value|unicode-events|paste-keycode` with text on stdin; one JSON line out; exit 2 on expected failure.

```
untype --api-key sk_... --language el --language en
SONIOX_API_KEY=sk_... untype --output-mode append
untype --stt-provider elevenlabs --elevenlabs-api-key xi_... --language auto
untype --no-refine --interaction-mode hybrid --protocol-output ./events.jsonl
```

05

Operating and troubleshooting

The daily loop, what the logs say, the symptoms you will meet,
and the rules that keep the two core features intact.



Focus a field, hold, speak, release.

01

Focus the target

Click into the field of any app. If untype is frontmost, the last external app is re-activated for you before delivery.

02

Hold and speak

The overlay shows the live partial and the R/T/C/I state. Press R, T, C or I to flip an operator mid-recording.

03

Release

The overlay moves to finalizing; with streaming on you see tokens arrive. The text lands at the caret and the overlay hides.

04

Use voice commands

In agent or hybrid mode, say "command send", "command cancel" or "command status", or the Greek guard phrase.

Quick Close trades accuracy for speed: it submits the latest partial the moment you release. Leave it off when the provider's final punctuation matters, turn it on for short agent commands.

One line tells you which link is slow or broken.

```
tail -1 ~/.tool-agents/untype/release-latency.jsonl | python3 -c \
  'import sys,json; r=json.load(sys.stdin); print(r.get("outcome"), r.get("focused_input"))'
# expected: delivered_to_focused_input {'attempted': True, 'ok': True, ...}
```

SIGNAL	INTERPRETATION
provider_final_wait_ms high	Provider finalization or voice activity detection is slow; consider Quick Close for short commands.
runtime_scheduling_ms high	UI or runtime scheduling contention on the main thread.
refine_ms / translate_ms high	The LLM is the bottleneck: model choice, reasoning effort, or network.
focused_input_ms high or ok=false	Accessibility, target application or paste problems; check the code field.
text_source=quick_close_partial with low total	Quick Close worked as intended.
text_source=timeout_fallback_partial	The provider did not finalize within 1.5 s; the latest partial was used with a warning.

What you see, why, and what to do.

SYMPTOM	CAUSE	FIX
<code>focused_input_failed</code> , <code>accessibility_not_trusted</code>	Accessibility grant tied to an older signing identity.	<code>tccutil reset Accessibility com.local.untype</code> , relaunch, re-add the app.
Hotkey does nothing after an install	Input Monitoring grant belongs to the previous identity.	Remove the stale entry, re-add <code>/Applications/untype.app</code> , relaunch.
Text processed but not inserted; untype was frontmost	No external target to restore.	Focus the target edit control before release.
Overlay stuck in finalizing for 15 s	Hung LLM request hitting the fixed timeout.	Check the endpoint and deployment; the section is delivered raw (fail-open).
Refinement silently skipped, HTTP 400	<code>UNTYPE_LLM_REASONING_EFFORT</code> set on a non-reasoning deployment.	Unset it for that deployment.
UI shows one Azure deployment, requests use another	Duplicate <code>AZURE_OPENAI_DEPLOYMENT</code> lines; last one wins.	Keep a single definition in <code>.env</code> .
Warm session keeps restarting after a provider error	Fatal provider error such as an exhausted balance.	Auto-restart is skipped after fatal errors; fix the account, restart the session.
Notarization <code>status: Invalid</code>	Signing or entitlement problem in the bundle.	<code>xcrun notarytool log <id> --keychain-profile untype-notary</code> .

Two features may never regress.

Do not break push-to-talk.

Any change to the UI, session controls, hotkey configuration, permission flow, overlay or runtime lifecycle must keep the whole chain: listening stays active, press and release reach the hotkey-owned session, the gate opens and closes, the overlay shows and hides, and delivery runs after release. Verify in

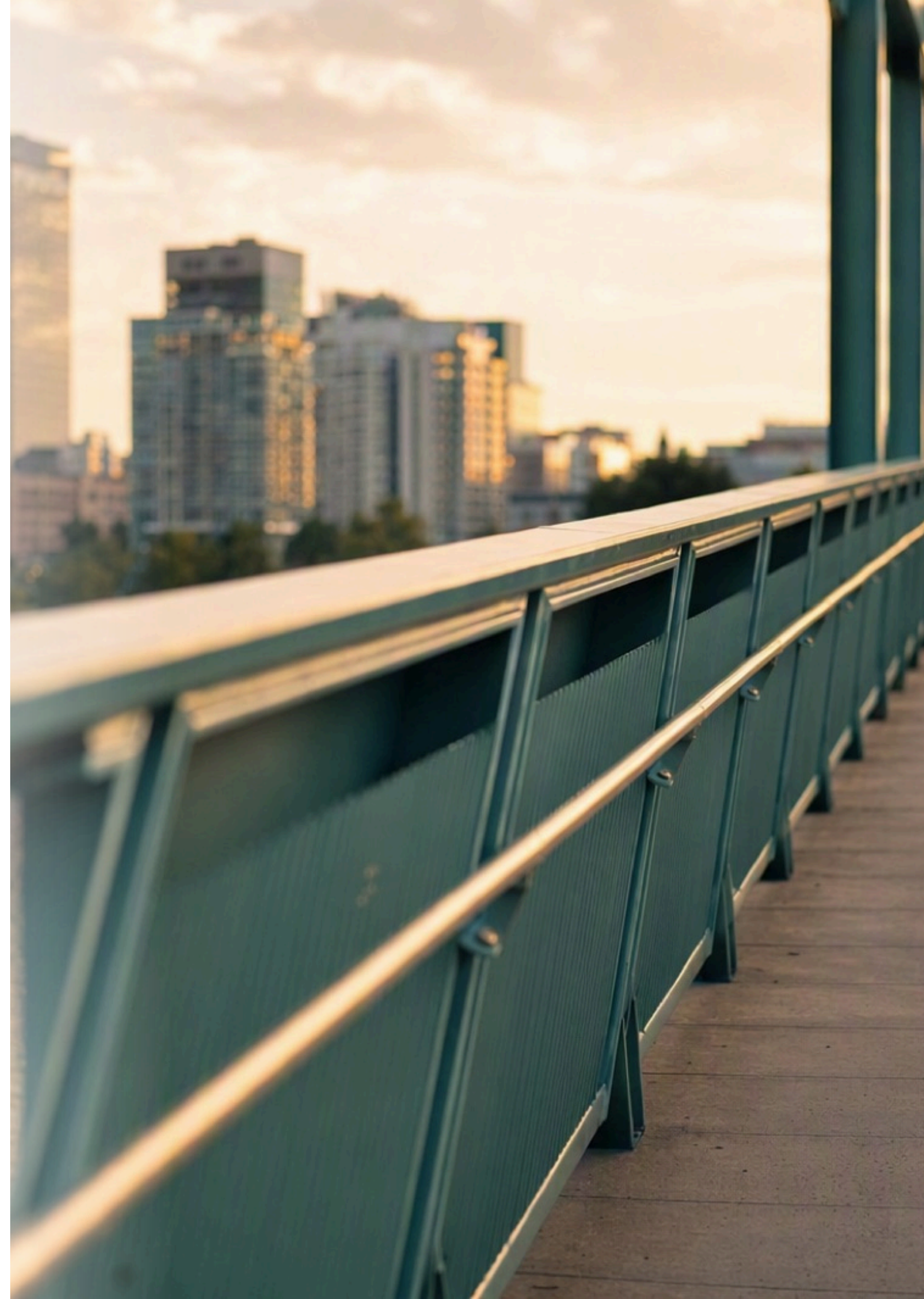
`NativeUntypeUILauncher`, `UntypeHotkeyMonitor`, `HotkeySessionControl` and `TranscriptionSessionRuntime`.

Do not break focused-input delivery.

Any change touching delivery, the protocol controller, the packaging script, or any redeploy must be followed by a real release into a text field with `focused_input.ok=true` in the log.

Never redeploy an ad-hoc bundle.

Always package with the Developer ID identity and notarize; an unsigned build silently revokes Accessibility.



untype

Where it stands, and what is next.

Build 7 ships signed and notarized with push-to-talk, focused-input delivery, refinement, translation and streaming in place.

Open items: synchronise runtime state across execution contexts, await provider handshakes and trailing finals, reduce UI main-thread writes, complete the visual verification pass, and replace the placeholder bundle id in one deliberate build.

Docs: `docs/design/project-design.md`,
`configuration-guide.md`, `deployment-guide.md`,
`release-runbook.md` and the issue register in
`Issues - Pending Items.md`.

